

NRT Hot Topics: Agentic AI — Lecture 2

Two-page reference: from AI models and LLM behavior to research-agent interfaces, MCP, permissions, and scoped subagents.

Core idea: Using an agent means delegating work to an LLM-driven system. Designing an agent means deciding what the AI should — and should not — have to figure out.

1. Model, workflow, tool

- **Model:** learned mapping from input to output.
- **Workflow:** preprocessing + model/algorithm + postprocessing + reporting.
- **Tool:** callable software interface to a model or workflow.

Research example: image → preprocess → segment/classify → postprocess → report.

2. Why repository-as-context can fail

- context dilution
- competing examples or aliases
- learned language priors
- semantic ambiguity
- silent, executable errors

20x may look plausible even when the tool requires 20.

Plausible continuation ≠ correct tool call.

3. Interface-design response

- Make parameter names explicit.
- Define allowed values/ranges.
- Validate before execution.
- Return structured results.

Design principle: Do not ask the LLM to infer what the software can specify explicitly.

4. When designing an agent is a good option

- repeated workflow across many samples/files
- specialized lab tools, scripts, or databases
- hard constraints on units, parameters, or outputs
- local/private data or permission boundaries matter
- results should be reusable, auditable, or shared across a team

Maybe not necessary for one-off, generic, low-stakes tasks when an existing GUI already works well.

Inputs



Parameters / Schema



Workflow



Structured Output

Stop & think: Pick one research task you already do. Which parts are true scientific judgment, and which parts could be made explicit as fields, constraints, validation rules, and structured outputs?

Stop & think: Would an agent need to call only the model, or is the *whole workflow* the safer and more useful capability to expose?

Research Agents: MCP, Permissions, and Scoped Delegation

5. MCP: a common protocol for callable capabilities

MCP helps an AI application discover and invoke selected external tools/resources through standardized interfaces. It does **not** replace the scientific computation itself.



6. Permissions

- **Allow:** execute without asking.
- **Ask:** require user approval.
- **Deny:** block the operation.

Permission rules move access control from model judgment into explicit configuration.

7. Scoped subagent

- specialized instructions
- scoped context
- approved tools
- permission boundary

Main agent coordinates; the subagent performs a bounded subtask and returns a structured result.

Permissions define *what can be accessed*; subagents define *who is delegated to work within that scope*.

Four-demo progression

Demo	Question to watch	Main idea
1. Repository-as-Context	What must the agent infer?	Interface reconstructed from files/context.
2. MCP Interface	What becomes explicit?	Declared capabilities, schema, structured results.
3. Permissions	What can the agent access?	Approved path remains available; direct access can be blocked.
4. Scoped Subagent	Who is delegated, and with what scope?	Specialized instructions + selected tools + permissions.

Helps with

- explicit parameters and schemas
- bounded structured outputs
- auditable tool calls
- minimizing unnecessary raw-data transfer
- limiting tools/data per subagent

Does not automatically guarantee

- scientific correctness
- data privacy / confidentiality
- correct authorization or safe permissions
- correct choice among valid options
- absence of model mistakes

Stop & think: What data should remain local in your workflow? What should the LLM see: raw data, derived summaries, file paths, or only final structured results?

Take-home framework: Inputs → Parameters → Workflow → Structured Output → Permissions + Human Review